Évolution Différentielle massivement parallèle sur GPU avec CUDA : trois versions comparées à la version séquentielle

Zainab Elamri, Omar Elsemry, Mohamed De Franceschi
Master IM, Université de Haute-Alsace (UHA)
Enseignant : Lhassane IDOUMGHAR

***Résumé* : Nous partons d'un PSO écrit en CUDA par notre professeur et nous le transformons en évolution différentielle (DE/rand/1/bin). Nous avons écrit une version séquentielle sur CPU et trois versions sur GPU : v1 avec un seul kernel et un thread par individu, v2 avec un kernel par étape (mutation, croisement, évaluation, sélection) et v3 avec un thread par coordonnée. Nous les testons sur Sphere, Rastrigin, Rosenbrock et Griewank décalées, avec $D \in \{10, 50, 100\}$, une population de 50, 100 ou 500 et un budget de $10^4 \times D$ évaluations. Sur une RTX 2080 Ti, les quatre versions donnent la même qualité (test de Wilcoxon) et battent nettement le PSO fourni. La v2 donne exactement les mêmes résultats que la v1 mais elle est en général plus lente. La v3 va jusqu'à 89 fois plus vite que le CPU, mais elle est plus lente que la v1 pour $D = 10$. Nous comparons aussi nos taux de succès à ceux de l'article de Qin et al. et nous testons $CR = 0{,}3$.**

***Mots-clés* : Évolution différentielle, GPU, CUDA, parallélisme, optimisation globale, fonctions de test.**

I. Introduction

On cherche le minimum global d'une fonction f sur un domaine borné, ici $[-5{,}12;\ 5{,}12]^D$, où D est le nombre de dimensions. Les fonctions qu'on teste ont beaucoup de minimums locaux (Rastrigin, Griewank) ou une vallée étroite (Rosenbrock), donc une méthode de descente classique reste vite bloquée. On utilise plutôt des métaheuristiques à population : on fait évoluer un groupe de solutions et on garde les meilleures. L'optimisation par essaim de particules (PSO) et l'évolution différentielle (DE) [2] en font partie.

Ces méthodes coûtent cher quand D et la taille de la population augmentent. Par contre, chaque individu subit les mêmes calculs, ce qui convient bien à un GPU, capable de faire tourner des milliers de threads en même temps [1].

Le professeur nous a donné un PSO écrit en CUDA. Nous l'avons d'abord étudié, puis nous l'avons transformé en DE (variante DE/rand/1/bin). Nous avons écrit une version séquentielle sur CPU, qui sert de référence, et trois versions GPU :

- v1 : un seul kernel fait toute une génération, avec un thread par individu ;
- v2 : un kernel par étape (mutation, croisement, évaluation, sélection), toujours avec un thread par individu ;
- v3 : un kernel par étape, mais avec un thread par coordonnée d'individu.

Nous comparons ces versions entre elles, avec le PSO fourni et avec les résultats de l'article de Qin et al. [1]. La section II explique les bases de la programmation GPU, la section III décrit le DE et nos implémentations, la section IV donne les résultats et la section V conclut.

II. Programmation GPU

Un CPU a peu de cœurs, mais chacun est rapide et sait gérer du code compliqué. Un GPU a des milliers de petits cœurs plus simples, faits pour appliquer le même calcul à beaucoup de données à la fois. CUDA est l'outil de NVIDIA pour programmer ces GPU en C/C++.

Un programme CUDA a deux parties. Le code *hôte* tourne sur le CPU : il réserve la mémoire du GPU (`cudaMalloc`), copie les données (`cudaMemcpy`) et lance des *kernels*. Un kernel est une fonction marquée `__global__` qui s'exécute sur le GPU. On la lance avec `kernel<<<blocs, threads>>>(...)` : le GPU crée alors une grille de blocs, et chaque bloc contient des threads. Chaque thread calcule son numéro avec `blockIdx.x * blockDim.x + threadIdx.x` et s'en sert pour savoir sur quelle donnée travailler. Les threads sont exécutés par paquets de 32 appelés *warps*. Les fonctions `__device__` ne peuvent être appelées que depuis le GPU.

Il y a plusieurs types de mémoire. Les registres sont propres à chaque thread. La mémoire partagée (`__shared__`) est rapide mais n'est visible que par les threads d'un même bloc ; `__syncthreads()` fait attendre tous les threads du bloc. La mémoire globale est visible par tous les threads, mais elle est plus lente. Enfin, les copies entre le CPU et le GPU passent par le bus PCIe et sont lentes, donc il faut en faire le moins possible.

Deux points nous ont servi pour la suite. D'abord, lancer un kernel n'est pas gratuit : cela prend quelques microsecondes, même si le kernel ne fait presque rien. Ensuite, deux kernels lancés l'un après l'autre par le CPU s'exécutent dans l'ordre : le second ne démarre que quand le premier a fini pour tous les threads. C'est le seul moyen simple de synchroniser tous les threads du GPU. Pour les nombres aléatoires, on utilise la bibliothèque cuRAND, avec un état de générateur (`curandState`) par thread.

Nos tests ont tourné sur une NVIDIA GeForce RTX 2080 Ti (architecture Turing, 68 multiprocesseurs, 11 Go) avec CUDA 13.4.

III. Évolution Différentielle

A. Version séquentielle

Le DE [2] fait évoluer une population de N_P individus. Chaque individu est un vecteur x_i de D coordonnées, tiré au hasard au départ. À chaque génération, on fait trois étapes pour chaque individu x_i :

1) Mutation : on tire trois autres individus r_1, r_2, r_3, différents entre eux et différents de i, et on calcule le mutant $v_i = x_{r_1} + F\,(x_{r_2} - x_{r_3})$.
2) Croisement : on construit l'essai u_i coordonnée par coordonnée. On prend $v_{i,j}$ si un nombre tiré au hasard est inférieur à CR, ou si $j = j_{\text{rand}}$, sinon on garde $x_{i,j}$. L'indice j_{rand} est tiré une fois par individu, pour être sûr qu'au moins une coordonnée vienne du mutant. On ramène ensuite u_i dans le domaine.
3) Sélection : si $f(u_i) \leq f(x_i)$, l'essai remplace x_i, sinon on garde x_i.

On prend $F = 0{,}5$ et $CR = 0{,}9$. Nous avons aussi testé $CR = 0{,}3$, la valeur de l'article [1]. Le sujet impose un budget de $10^4 \times D$ évaluations de f. Une génération coûte N_P évaluations, donc on fait $10^4 \times D/N_P$ générations : par exemple 2000 générations pour $D = 10$ et $N_P = 50$, mais seulement 200 pour $N_P = 500$.

Les mutants de la génération g doivent être construits à partir de la population de la génération g. On lit donc dans un tableau et on écrit dans un autre, puis on échange les deux pointeurs (double buffer). La version CPU fait la même chose, avec une simple boucle sur les individus.

B. Version massivement parallèle

1) Le PSO fourni par le professeur: Avant d'écrire le DE, nous avons lu le PSO CUDA du professeur. `main.cpp` tire les positions au hasard sur le CPU et appelle `cuda_pso`. Cette fonction copie les tableaux sur le GPU, puis répète `MAX_ITER` fois :

- `kernelUpdateParticle` : un thread par *coordonnée* ($N_P \times D$ threads). Le thread i met à jour la vitesse puis la position de la case i du tableau ; il retrouve la coordonnée du meilleur global avec `gBest[i % D]`. Les deux nombres aléatoires r_1 et r_2 sont tirés sur le CPU et passés en paramètre, donc ils sont les mêmes pour toutes les particules d'une itération ;
- `kernelUpdatePBest` : seuls les threads avec `i % D == 0` travaillent, donc un par particule. Chacun compare sa position à son meilleur personnel et le remplace s'il est meilleur ;
- ensuite tous les meilleurs personnels sont recopiés sur le CPU, qui cherche le meilleur global avec une boucle, puis le renvoie au GPU.

Il y a donc deux copies CPU↔GPU et une boucle CPU à chaque itération, ce qui ralentit beaucoup le programme.

TABLE I
ERREURS TROUVÉES DANS LE CODE FOURNI.

Problème	Corrigé	Raison
Griewank : produit qui commence à 0	oui	la fonction testée était fausse
Dim, Pop, fonction en constantes	oui	nécessaire pour faire tous les tests
`MAX_ITER` en itérations	oui	même budget d'évaluations pour tous
`getRandom` : départ jusqu'à 6,12	non	même départ pour toutes les versions
Levy : terme dans la boucle	non	fonction non testée
PSO : nombre de blocs arrondi	non	PSO gardé comme référence
PSO : `tempParticle` partagés	non	PSO gardé comme référence
`float` + constante dans f	non	fonctions du sujet gardées (section IV-B)

2) Erreurs trouvées dans le code fourni: En lisant le code, nous avons trouvé plusieurs erreurs. Le tableau I les résume. Nous avons corrigé celles qui faussaient nos résultats, et laissé les autres pour garder le code fourni le plus proche possible de l'original.

La plus grave est dans Griewank : le produit des cosinus commençait à 0 au lieu de 1. Il restait donc toujours nul, et la fonction calculée n'était pas Griewank mais presque une Sphere. Nous l'avons corrigée dans la version CPU et dans la version GPU de la fonction, y compris dans le PSO.

Deux autres points ne sont pas vraiment des erreurs, mais nous avons dû les adapter au sujet. Dim, Pop et la fonction étaient des constantes écrites dans `kernel.h` ; nous les avons rendues réglables à la compilation, avec les mêmes valeurs par défaut. Et `MAX_ITER` compte des itérations, alors que le sujet impose un nombre d'évaluations : nous donnons donc au PSO $10^4 \times D/N_P$ itérations, ce qui fait le même nombre d'évaluations que le DE.

Nous avons laissé les autres erreurs. `getRandom` tire les positions de départ entre $-5{,}12$ et $6{,}12$ au lieu de $5{,}12$, à cause d'un `+1` en trop. Comme toutes nos versions utilisent cette fonction avec la même graine, elles partent des mêmes populations, et le DE ramène chaque essai dans le domaine dès la première génération. Dans Levy, un terme est ajouté à chaque tour de boucle au lieu d'une seule fois, mais cette fonction ne fait pas partie des tests demandés. Dans le PSO, le nombre de blocs `ceil(size / threadsNum)` est calculé avec une division entière : quand $N_P \times D$ n'est pas un multiple de 32, ce qui arrive dans tous nos tests, les dernières coordonnées ne sont jamais mises à jour. De plus, les tableaux `tempParticle1` et `tempParticle2` sont globaux : tous les threads y écrivent leur particule en même temps, donc un thread peut évaluer la particule d'un autre. Nous n'avons pas corrigé le PSO, car il sert de référence "telle que fournie". Il faut donc garder en tête que nos résultats le désavantagent. Enfin, la fonction est calculée en `float` avec la constante (-450, etc.) ajoutée à l'intérieur, ce qui limite la précision (voir section IV-B).

3) v1 : un seul kernel: Pour le DE, nous avons gardé la même fonction `cuda_pso` pour ne pas toucher à `main.cpp`. Toute la population reste sur le GPU du début à la fin : une copie au début, une copie à la fin. Le kernel `kernelDEGeneration` est lancé une fois par génération avec un thread par individu. Chaque thread tire r_1, r_2, r_3, construit son essai, l'évalue et fait la sélection. Comme dans la version CPU, on lit dans `popIn` et on écrit dans `popOut`, puis le CPU échange les deux pointeurs. Sans cela, un thread pourrait remplacer un individu pendant qu'un autre thread le lit pour sa mutation.

4) v2 : un kernel par étape: Dans la v2, chaque étape a son propre kernel, comme dans la version de base de l'article [1] (figure 2). À chaque génération, le CPU lance cinq kernels l'un après l'autre :

- (P) tire r_1, r_2, r_3 et j_{rand} et les range dans un tableau ;
- (M) calcule les mutants dans le tableau `mutant` ;
- (C) fait le croisement et écrit les essais dans `trial` ;
- (E) évalue les essais (c'est le même kernel que pour la population de départ) ;
- (S) remplace x_i par u_i quand l'essai est meilleur.

Les kernels se passent leurs résultats par la mémoire globale. Le double buffer n'est plus nécessaire : quand (S) démarre, (M) est déjà fini pour tous les individus, donc personne ne lit plus la population. On peut écrire directement dedans. Les nombres aléatoires sont tirés dans le même ordre que dans la v1, et nous avons vérifié que la v2 donne exactement les mêmes résultats que la v1 sur les 360 runs. Le code est plus facile à lire, mais il y a cinq lancements de kernel par génération au lieu d'un, et les tableaux intermédiaires font des allers-retours en mémoire globale.

5) v3 : un thread par coordonnée: Avec un thread par individu, une population de 50 n'occupe que 50 threads alors que le GPU peut en faire tourner des dizaines de milliers. Et chaque thread fait une boucle sur les D coordonnées. Dans la v3, nous gardons les mêmes kernels que la v2, mais chaque thread s'occupe d'une seule coordonnée. Avec 100 individus en 3 dimensions (x, y, z), on lance 300 threads au lieu de 100 (figure 3). Le thread k traite l'individu $i = k/D$ et la coordonnée $d = k \bmod D$, c'est-à-dire la case k du tableau de la population. C'est l'idée de la version améliorée de l'article (cudaDEi), et c'est aussi ce que fait déjà `kernelUpdateParticle` dans le PSO fourni. Ce changement nous a obligés à modifier plusieurs choses :

1) Les indices r_1, r_2, r_3 et j_{rand} doivent être les mêmes pour toutes les coordonnées d'un individu. Le kernel (P) garde donc un thread par individu et range les indices en mémoire globale. Les autres kernels les relisent.
2) Pour le croisement, chaque coordonnée tire son propre nombre aléatoire. Il faut donc un état cuRAND par coordonnée, soit $N_P \times D$ états au lieu de N_P.
3) L'évaluation est la partie la plus difficile. f est une somme sur les coordonnées (plus un produit pour Griewank), donc un thread ne peut pas la calculer seul. Nous avons réécrit la fonction objectif en une fonction `terme` qui calcule la part d'une seule coordonnée. Pour Rosenbrock, le terme d a besoin de la coordonnée $d + 1$, que le thread va lire en mémoire globale. Ensuite, il faut additionner les D termes. Pour cela, tous les threads d'un même individu sont placés dans le même bloc et écrivent leur terme en mémoire partagée. On fait alors une réduction : à chaque tour, la première moitié des threads ajoute la case de la seconde moitié, avec `__syncthreads()` entre deux tours, jusqu'à ce qu'il ne reste qu'une case. Comme cette méthode demande une puissance de 2, le bloc a une largeur de 16, 64 ou 128 pour $D = 10$, 50 ou 100, et les threads en trop ajoutent 0 (ou multiplient par 1 pour le produit). Pour ne pas avoir des blocs trop petits, un bloc traite plusieurs individus : 8 pour $D = 10$, 2 pour $D = 50$ et 1 pour $D = 100$, soit toujours 128 threads.
4) Pour la sélection, chaque thread recopie sa coordonnée si l'essai est meilleur, et le thread de la coordonnée 0 met à jour la fitness. Mais si ce thread écrit la nouvelle fitness alors que les autres threads du même individu ne l'ont pas encore lue, ils comparent avec une mauvaise valeur. On lit donc la fitness dans un tableau et on l'écrit dans un autre, échangés à chaque génération.

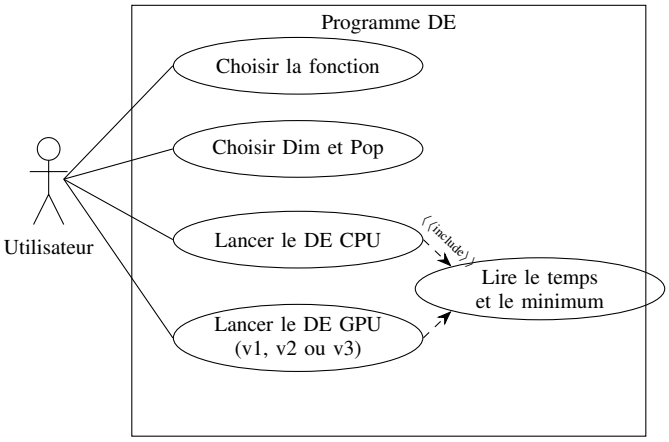


FIGURE 1. Diagramme de cas d'utilisation. Dim, Pop et la fonction sont fixés à la compilation (`make gpu DIM=10 POP=100 OBJ_FUNC=4`).

Comme les nombres aléatoires ne sont plus tirés dans le même ordre, la v3 ne donne pas exactement les mêmes résultats que la v1. On vérifie avec des tests statistiques que la qualité reste la même (section IV). Cette version a deux limites : D doit rester inférieur ou égal à 1024 (nombre maximum de threads par bloc) et, pour $D = 10$, 6 threads sur 16 ne servent à rien pendant l'évaluation.

IV. RÉSULTATS EXPÉRIMENTAUX

A. Configuration

Machine : processeur Intel Core i7-8700K (3,7 GHz), carte NVIDIA RTX 2080 Ti, Arch Linux, CUDA 13.4, GCC 15. Tout est compilé en `-O2`, et pour `sm_75` côté GPU.

Nous comparons cinq programmes : le DE séquentiel (CPU), les DE GPU v1, v2 et v3, et le PSO du professeur ($\omega =$

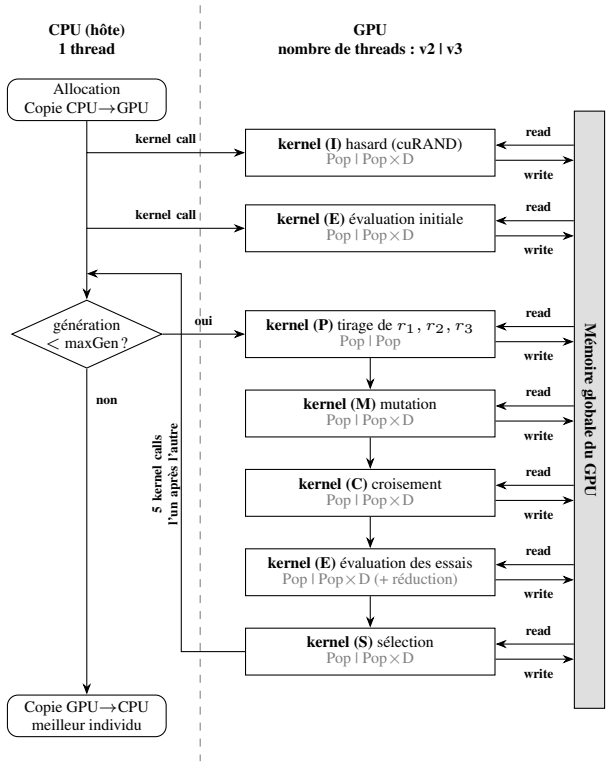


FIGURE 2. Organigramme du DE massivement parallèle avec un kernel par étape (versions v2 et v3). En gris, le nombre de threads lancés par chaque kernel. Dans la v1, les kernels (P), (M), (C), (E) et (S) ne forment qu'un seul kernel.

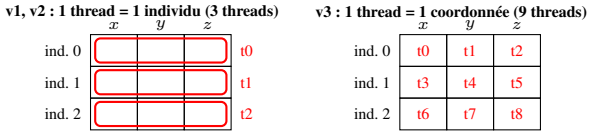


FIGURE 3. Répartition du travail avec 3 individus et 3 dimensions. Le thread k de la v3 traite l'individu $i = k/D$ et la coordonnée $d = k \bmod D$.

0,5, $c_1 = c_2 = 1{,}5$). Le sujet demande $N_P \in \{50, 100, 500\}$, $D \in \{10, 50, 100\}$, un budget de $10^4 \times D$ évaluations et 10 runs par cas. Avec les 4 fonctions, cela fait 36 cas par programme, donc 1800 runs. Nous avons aussi refait le DE CPU et le DE GPU v1 avec $CR = 0{,}3$ (720 runs de plus).

`main.cpp` ne fait qu'un run et affiche la fitness en `float`. Nous avons donc écrit un petit programme de test (`bench_main.cpp`) qui appelle la même fonction `cuda_pso`, fait les 10 runs et calcule l'erreur en double précision. La graine est fixée (12345), donc les cinq programmes partent des mêmes populations de départ. Le temps mesuré est le temps réel de l'appel à `cuda_pso`, avec les allocations et les copies. Un premier run, qui sert à initialiser CUDA, n'est pas compté. Dim, Pop et la fonction sont choisis à la compilation (`make dim DIM=50 POP=100 OBJ_FUNC=1`), car le code fourni utilise des constantes pour la taille des tableaux.

Pour chaque cas, on donne la moyenne et l'écart-type de l'erreur $f(x_{\text{meilleur}}) - f(x^\star)$ sur les 10 runs. Pour savoir si deux programmes donnent des résultats vraiment différents, on utilise le test de Wilcoxon (somme des rangs) et le test de Kruskal-Wallis avec un seuil de 5 % : si $p < 0{,}05$, on considère que la différence n'est pas due au hasard.

B. Fonctions de test

On utilise les quatre fonctions demandées, en version décalée [3], avec $z = x - o$. Dans le code fourni le décalage o vaut 0, donc $z = x$:

$$f_{\text{Sphere}} = \sum_{i=1}^{D} z_i^2 - 450,$$

$$f_{\text{Rastrigin}} = \sum_{i=1}^{D} \left(z_i^2 - 10\cos(2\pi z_i) + 10\right) - 330,$$

$$f_{\text{Rosenbrock}} = \sum_{i=1}^{D-1} \left(100(z_i^2 - z_{i+1})^2 + (z_i - 1)^2\right) + 390,$$

$$\text{avec ici } z = x + 1,$$

$$f_{\text{Griewank}} = \sum_{i=1}^{D} \frac{z_i^2}{4000} - \prod_{i=1}^{D} \cos\frac{z_i}{\sqrt{i}} + 1 - 180.$$

Le minimum est en $x = 0$ et vaut -450, -330, 390 et -180. Sphere est la plus facile : une seule "cuvette". Rastrigin et Griewank ont énormément de minimums locaux. Rosenbrock a un seul minimum, mais au fond d'une vallée longue et plate où l'algorithme avance lentement.

Les calculs sont en `float`. Comme on ajoute -450 (ou une autre constante) au résultat, deux fitness ne peuvent plus être distinguées si elles diffèrent de moins de 3×10^{-5} environ. C'est pourquoi l'erreur ne descend presque jamais sous 10^{-5} dans nos tableaux. Nous n'avons pas passé les calculs en `double` pour deux raisons : nous voulions garder les fonctions du code fourni, et sur notre carte grand public les calculs en `double` sont environ 32 fois plus lents qu'en `float`, ce qui aurait faussé les mesures de temps. La vraie solution serait de calculer f sans la constante et de ne l'ajouter qu'à l'affichage : on pourrait alors rester en `float`.

C. Résultats

1) Qualité des solutions: Les tableaux III à VI donnent l'erreur moyenne pour tous les cas. Les quatre versions du DE ont la même qualité. La v2 donne exactement les mêmes nombres que la v1, ce qui est normal puisqu'elle fait les mêmes calculs dans le même ordre. Le test de Wilcoxon ne trouve une différence entre le CPU et la v1 que dans 2 cas sur 36, entre le CPU et la v3 dans aucun cas, et entre la v1 et la v3 dans 2 cas. Le test de Kruskal-Wallis sur les quatre versions trouve une différence dans 2 cas sur 36. Avec un seuil de 5 %, on s'attend à environ 2 cas sur 36 juste par hasard. Nos versions GPU ne changent donc pas le résultat de l'algorithme, elles le calculent juste plus vite.

À budget égal, le DE fait beaucoup mieux que le PSO fourni. La différence est significative dans 34 cas sur 36, et la v3 a une erreur moyenne plus faible dans 33 cas. Par exemple, pour Sphere avec $D = 100$ et Pop 100, l'erreur est

de l'ordre de 10^{-5} pour le DE et de 114 pour le PSO. Le PSO n'est meilleur que sur Rastrigin avec Pop 500, où toutes les méthodes restent loin du minimum. Les défauts du PSO cités en section III jouent probablement un rôle, mais nous ne l'avons pas vérifié.

Le budget est fixe, donc une grande population veut dire peu de générations : 200 pour $D = 50$ et Pop 500. Dans ce cas l'algorithme n'a pas le temps de converger. Pour Sphere avec $D = 50$, l'erreur passe de 10^{-5} avec Pop 100 à 0,2 avec Pop 500. Pour Rastrigin, la meilleure qualité est presque toujours obtenue avec Pop 50. Une grande population fait gagner du temps sur GPU, mais pas de la qualité.

2) Temps de calcul: Le tableau VII donne le temps moyen, le tableau VIII l'accélération par rapport au CPU et les figures 4 et 5 les courbes de temps. Le temps du CPU ne dépend presque pas de la population, car il fait toujours le même nombre d'évaluations. Sur GPU, au contraire, le temps baisse quand la population augmente : il y a moins de générations, donc moins de lancements de kernel, et plus de threads travaillent en même temps. Le PSO est le plus lent partout (jusqu'à 18 s pour $D = 100$), à cause de ses copies CPU↔GPU à chaque itération.

La v1 va jusqu'à 27 fois plus vite que le CPU (médiane 3,5 sur les 36 cas). Elle est plus lente que le CPU dans 3 cas avec $D = 50$ et Pop 50 ou 100 : seulement 50 ou 100 threads travaillent, et chacun fait seul toute la boucle sur les coordonnées.

La v2 est en général plus lente que la v1 : 1,4 fois plus lente en médiane, et jusqu'à 3 fois. Elle est plus lente que le CPU dans 6 cas, tous avec Pop 50. Le tableau II montre le temps de chaque kernel. Pour $D = 10$ et Pop 50, chaque kernel prend environ 3 à 5 μs, quel que soit son travail : c'est surtout le coût du lancement. La v2 lance 5 kernels par génération au lieu d'un, sur 2000 générations. Pour $D = 100$, les kernels (M), (C) et (E) de la v2 sont longs (18 à 37 μs), car chaque thread fait une boucle de 100 tours et relit en mémoire globale les tableaux écrits par le kernel d'avant. Il y a aussi une raison connue sur GPU : quand les 32 threads d'un warp lisent des cases voisines en mémoire, le GPU les charge en une seule fois. Dans la v1 et la v2, le thread i lit la case $i \times D + d$, donc deux threads voisins lisent des cases éloignées de D cases, et le GPU doit faire beaucoup plus de lectures. Pour $D = 50$, la v1 et la v2 ont presque le même temps ; nous n'avons pas cherché plus loin pourquoi.

La v3 est la plus rapide dès que $D \geq 50$: jusqu'à 89 fois plus rapide que le CPU (médiane 8,4 sur les 36 cas), et 2,4 à 6,8 fois plus rapide que la v1 pour $D = 50$ et $D = 100$. Ici, des threads voisins lisent des cases voisines, et il y a $N_P \times D$ threads, par exemple 50 000 pour Pop 500 et $D = 100$, ce qui occupe bien le GPU. Dans le tableau II, tous ses kernels restent entre 3 et 5 μs environ, même pour $D = 100$. Par contre, pour $D = 10$, la v3 est plus lente que la v1 (environ 1,5 fois). Chaque kernel a très peu de travail, donc le temps est surtout celui des 5 lancements par génération, contre un seul pour la v1. C'est pour cela que l'article regroupe les kernels de la boucle en un seul (kernel MCER de cudaDEi [1]) tout en gardant un thread par coordonnée.

TABLE II
TEMPS MOYEN D'UN LANCEMENT DE CHAQUE KERNEL (μS), RASTRIGIN. MESURÉ AVEC `cudaEvent` (OPTION `-DPROFIL`) ; LA MESURE AJOUTE UNE ATTENTE APRÈS CHAQUE KERNEL, DONC CES TEMPS SERVENT SEULEMENT À COMPARER LES KERNELS ENTRE EUX.

Cas	Version	(P)	(M)	(C)	(E)	(S)	Total
D=10, Pop 50	v2	3,0	3,4	4,2	4,6	3,0	18,2
(2000 gén.)	v3	3,3	2,7	3,3	3,0	2,8	15,1
D=100, Pop 100	v2	3,1	18,1	20,7	36,5	3,7	82,1
(10 000 gén.)	v3	3,7	2,9	4,6	3,3	2,8	17,3

3) Comparaison avec l'article: L'article [1] compte un succès quand l'erreur est inférieure à 10^{-8}. Avec nos calculs en `float`, on n'arrive jamais à ce seuil, même sur Sphere (voir section IV-B). Dans le tableau IX, nous utilisons donc un seuil de 10^{-4}, plus facile à atteindre : la comparaison est à notre avantage. Les temps de l'article ne sont pas comparés, car leur carte graphique (GTX 560) est beaucoup plus ancienne.

Avec $CR = 0{,}9$, nous avons le même taux que l'article (1,00) pour Sphere et Griewank avec $D = 50$ ou 100 et Pop 50 ou 100 (sauf Griewank, $D = 100$, Pop 50 : 0,90). Nous sommes moins bons pour Griewank avec $D = 10$ (0,20 et 0,80 au lieu de 1,00), pour Rastrigin avec $D = 10$ (0,50 puis 0 au lieu de 1,00) et pour Pop 500, où notre taux est souvent nul. Sur Rosenbrock, les deux sont faibles. Nous obtenons 1,00 avec $D = 10$ et Pop 100, contre 0,32 dans l'article, mais 0 avec Pop 50, contre 0,44.

4) Essai d'amélioration : $CR = 0{,}3$: Le sujet demande d'essayer d'améliorer la qualité. L'article utilise $CR = 0{,}3$ alors que nous avions pris $CR = 0{,}9$. Nous avons donc relancé le DE CPU et la v1 avec $CR = 0{,}3$, sans rien changer d'autre (tableaux IX et X). Le résultat dépend beaucoup de la fonction : $CR = 0{,}3$ est meilleur dans 19 cas sur 36 et moins bon dans 17. Il aide surtout les grandes populations. Avec Pop 500, Sphere $D = 50$ passe d'une erreur de 0,21 à $4{,}3 \times 10^{-5}$ et Griewank $D = 50$ et $D = 100$ passent d'un taux de succès de 0 à 1,00. Rastrigin $D = 10$ passe aussi à 1,00 avec Pop 50 et 100, comme dans l'article. Par contre, Rosenbrock $D = 10$, Pop 100 passe de 1,00 à 0, et Rastrigin avec Pop 50 devient bien moins bon pour $D = 50$ (erreur de 34 à 146) et $D = 100$ (de 95 à 554). Une explication possible, que nous n'avons pas vérifiée : avec un CR faible, l'essai ne change que peu de coordonnées à la fois. Cela marche bien quand les variables sont indépendantes (Sphere, Rastrigin), mais moins bien sur Rosenbrock où chaque variable dépend de la suivante. Avec $CR = 0{,}3$, le GPU reste équivalent au CPU (différence significative dans 1 cas sur 36).

V. CONCLUSION

Nous avons étudié le PSO CUDA fourni, puis nous l'avons transformé en DE/rand/1/bin. Nous avons écrit une version CPU et trois versions GPU. Les quatre versions donnent des solutions de même qualité (tests de Wilcoxon et de Kruskal-

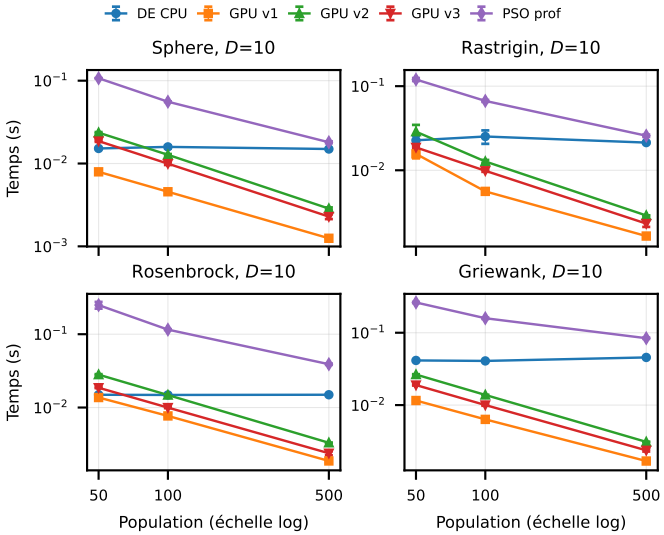


FIGURE 4. Temps moyen d'un run selon la population, $D = 10$ (échelles logarithmiques, barres : écart-type sur 10 runs).

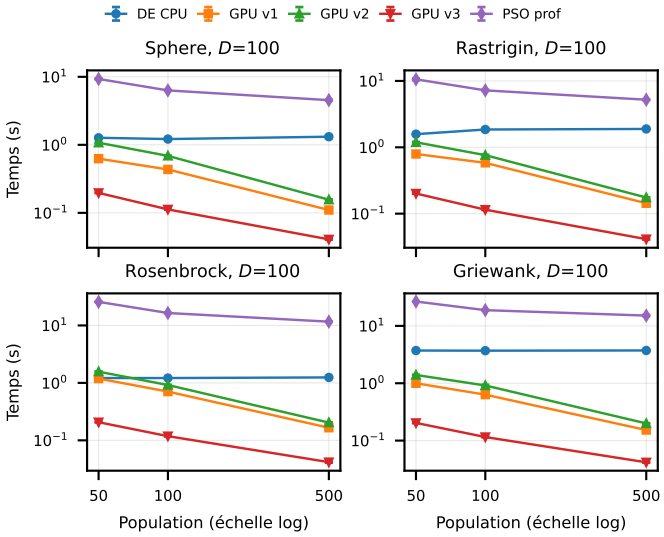


FIGURE 5. Temps moyen d'un run selon la population, $D = 100$.

TABLE III
SPHERE : ERREUR MOYENNE (MOYENNE ± ÉCART-TYPE SUR 10 RUNS POUR LE DE CPU, MOYENNE SEULE POUR LES AUTRES ; PLUS PETIT = MEILLEUR).

D	Pop	DE CPU	GPU v1	GPU v2	GPU v3	PSO
10	50	1.2e-5±1.8e-6	1.1e-5	1.1e-5	1.2e-5	0.791
10	100	1.1e-5±2.5e-6	9.9e-6	9.9e-6	1.1e-5	1.12
10	500	1.0e-5±3.6e-6	1.1e-5	1.1e-5	9.4e-6	0.393
50	50	1.4e-5±2.0e-6	1.3e-5	1.3e-5	1.3e-5	46
50	100	1.3e-5±2.3e-6	1.4e-5	1.4e-5	1.3e-5	33.9
50	500	0.174±0.0195	0.208	0.208	0.198	37.1
100	50	3.3e-5±1.4e-5	2.5e-5	2.5e-5	3.8e-5	122
100	100	3.0e-5±1.4e-5	3.7e-5	3.7e-5	2.3e-5	114
100	500	0.173±0.0274	0.184	0.184	0.178	133

TABLE IV
RASTRIGIN : ERREUR MOYENNE (MOYENNE ± ÉCART-TYPE SUR 10 RUNS POUR LE DE CPU, MOYENNE SEULE POUR LES AUTRES ; PLUS PETIT = MEILLEUR).

D	Pop	DE CPU	GPU v1	GPU v2	GPU v3	PSO
10	50	0.959±1.3	0.504	0.504	1.85	23.5
10	100	18.5±3.91	17.5	17.5	18.9	27.5
10	500	28.2±4.57	25.2	25.2	28.4	16.6
50	50	30±5.51	33.5	33.5	33.4	382
50	100	173±38.1	243	243	184	331
50	500	420±14	421	421	420	325
100	50	99.7±13.2	95.4	95.4	90	896
100	100	125±106	128	128	86.4	864
100	500	894±24.1	905	905	902	831

TABLE V
ROSENBROCK : ERREUR MOYENNE (MOYENNE ± ÉCART-TYPE SUR 10 RUNS POUR LE DE CPU, MOYENNE SEULE POUR LES AUTRES ; PLUS PETIT = MEILLEUR).

D	Pop	DE CPU	GPU v1	GPU v2	GPU v3	PSO
10	50	3.55±1.28	3.14	3.14	3.61	191
10	100	1.1e-5±3.2e-6	1.0e-5	1.0e-5	9.0e-4	311
10	500	3.37±0.345	3.12	3.12	3.3	37.1
50	50	56.5±31.1	46.9	46.9	41.1	4.5e4
50	100	20.4±1.96	20.7	20.7	19.9	2.5e4
50	500	175±22.1	164	164	162	2.8e4
100	50	137±59.5	122	122	127	1.2e5
100	100	99.9±25.3	111	111	105	1.6e5
100	500	207±29.6	212	212	201	1.4e5

TABLE VI
GRIEWANK : ERREUR MOYENNE (MOYENNE ± ÉCART-TYPE SUR 10 RUNS POUR LE DE CPU, MOYENNE SEULE POUR LES AUTRES ; PLUS PETIT = MEILLEUR).

D	Pop	DE CPU	GPU v1	GPU v2	GPU v3	PSO
10	50	7.7e-3±0.0109	0.0102	0.0102	7.0e-3	0.134
10	100	2.9e-3±9.1e-3	5.5e-3	5.5e-3	5.4e-6	0.173
10	500	0.0324±0.0183	0.033	0.033	0.0321	0.0324
50	50	6.7e-6±9.3e-7	6.2e-6	6.2e-6	6.3e-6	0.839
50	100	7.0e-6±6.5e-7	6.6e-6	6.6e-6	7.5e-4	0.618
50	500	5.5e-3±1.1e-3	6.1e-3	6.1e-3	6.4e-3	0.701
100	50	8.5e-6±3.5e-6	7.5e-4	7.5e-4	7.5e-4	0.872
100	100	1.2e-5±6.8e-6	1.8e-5	1.8e-5	8.8e-6	0.884
100	500	3.2e-3±4.8e-4	3.5e-3	3.5e-3	3.2e-3	0.881

TABLE VII
TEMPS MOYEN D'UN RUN (S), MOYENNE SUR LES 4 FONCTIONS (40 RUNS PAR CASE).

D	Pop	DE CPU	GPU v1	GPU v2	GPU v3	PSO
10	50	0.024	0.012	0.027	0.019	0.185
10	100	0.024	0.006	0.014	0.010	0.099
10	500	0.024	0.002	0.003	0.002	0.042
50	50	0.478	0.389	0.382	0.095	4.567
50	100	0.499	0.222	0.221	0.052	3.071
50	500	0.502	0.052	0.049	0.015	2.092
100	50	1.945	0.902	1.307	0.202	18.047
100	100	1.996	0.589	0.821	0.115	12.201
100	500	2.046	0.143	0.184	0.041	9.128

Wallis), et le DE est bien meilleur que le PSO fourni à budget

TABLE VIII
ACCÉLÉRATION PAR RAPPORT AU DE SÉQUENTIEL ($T_{\mathrm{CPU}}/T_{\mathrm{GPU}}$), MOYENNE SUR LES 4 FONCTIONS. FORME *v1 / v2 / v3*.

D	Pop 50	Pop 100	Pop 500
10	1.9/0.9/1.3	4.0/1.8/2.4	15.0/8.0/10.3
50	1.2/1.3/5.0	2.2/2.3/9.6	9.6/10.2/33.7
100	2.2/1.5/9.6	3.4/2.4/17.3	14.3/11.1/49.4

TABLE IX
TAUX DE SUCCÈS, FORME *article / nous avec* CR=0,9 */ nous avec* CR=0,3 (DE GPU v1). ARTICLE [1] : ERREUR $< 10^{-8}$, 25 RUNS. NOUS : ERREUR $< 10^{-4}$, 10 RUNS.

Fonction	D	Pop 50	Pop 100	Pop 500
Sphere	10	1.00/1.00/1.00	1.00/1.00/1.00	1.00/1.00/1.00
Sphere	50	1.00/1.00/1.00	1.00/1.00/1.00	1.00/0.00/1.00
Sphere	100	1.00/1.00/1.00	1.00/1.00/1.00	1.00/0.00/0.00
Rastrigin	10	1.00/0.50/1.00	1.00/0.00/1.00	1.00/0.00/0.00
Rastrigin	50	0.00/0.00/0.00	0.04/0.00/0.00	0.04/0.00/0.00
Rastrigin	100	0.00/0.00/0.00	0.00/0.00/0.00	0.00/0.00/0.00
Rosenbrock	10	0.44/0.00/0.00	0.32/1.00/0.00	0.00/0.00/0.00
Rosenbrock	50	0.00/0.00/0.00	0.04/0.00/0.00	0.00/0.00/0.00
Rosenbrock	100	0.00/0.00/0.00	0.00/0.00/0.00	0.00/0.00/0.00
Griewank	10	1.00/0.20/0.90	1.00/0.80/1.00	1.00/0.00/0.00
Griewank	50	0.92/1.00/1.00	1.00/1.00/1.00	1.00/0.00/1.00
Griewank	100	0.84/0.90/1.00	0.96/1.00/1.00	0.96/0.00/1.00

TABLE X
DE GPU v1 : ERREUR MOYENNE, FORME CR=0,9 / CR=0,3 (10 RUNS).

Fonction	D	Pop 50	Pop 100	Pop 500
Sphere	10	1.1e-5/1.2e-5	9.9e-6/1.1e-5	1.1e-5/1.2e-5
Sphere	50	1.3e-5/3.9e-5	1.4e-5/4.2e-5	0.208/4.3e-5
Sphere	100	2.5e-5/4.4e-5	3.7e-5/4.8e-5	0.184/5.0e-4
Rastrigin	10	0.504/1.1e-5	17.5/1.1e-5	25.2/5.73
Rastrigin	50	33.5/146	243/173	421/238
Rastrigin	100	95.4/554	128/599	905/719
Rosenbrock	10	3.14/2.65	1.0e-5/4.33	3.12/7.23
Rosenbrock	50	46.9/38.1	20.7/40.3	164/47.2
Rosenbrock	100	122/84.4	111/88.3	212/336
Griewank	10	0.0102/7.5e-4	5.5e-3/6.2e-6	0.033/5.5e-4
Griewank	50	6.2e-6/2.0e-5	6.6e-6/2.0e-5	6.1e-3/2.1e-5
Griewank	100	7.5e-4/2.1e-5	1.8e-5/2.3e-5	3.5e-3/4.7e-5

égal.

Pour la vitesse, séparer les étapes en plusieurs kernels (v2) rend le code plus clair mais plus lent, à cause des lancements de kernel en plus et des tableaux qui passent par la mémoire globale. Mettre un thread par coordonnée (v3) est le changement qui compte le plus : la v3 va jusqu'à 89 fois plus vite que le CPU et jusqu'à 6,8 fois plus vite que la v1 quand $D \geq 50$. Pour $D = 10$, elle reste plus lente que la v1, car il y a trop peu de travail par kernel. Enfin, $CR = 0{,}3$ améliore les grandes populations et Rastrigin en $D = 10$, mais dégrade Rosenbrock.

Notre travail a des limites. Comme la fonction est calculée en `float` avec la constante (-450, etc.) ajoutée à l'intérieur, nous ne pouvons pas atteindre une erreur de 10^{-8}, ni donc comparer nos taux de succès à ceux de l'article avec le même seuil. Le PSO a été gardé avec ses défauts. Nous n'avons testé qu'une seule autre valeur de CR. Pour aller plus loin, on pourrait regrouper (M), (C), (E) et (S) en un seul kernel tout en gardant un thread par coordonnée, comme cudaDEi, ce qui devrait régler le problème de la v3 pour $D = 10$. On pourrait aussi calculer en double précision et essayer d'autres stratégies, comme DE/best/1 ou un CR qui s'adapte pendant le run.

RÉFÉRENCES

[1] A. K. Qin, F. Raimondo, F. Forbes, and Y. S. Ong, "An improved CUDA-based implementation of differential evolution on GPU," in *Proc. GECCO'12*, Philadelphia, USA, 2012.

[2] K. Price, R. Storn, and J. Lampinen, *Differential Evolution : A Practical Approach to Global Optimization*. Springer, 2005.

[3] P. N. Suganthan *et al.*, "Problem definitions and evaluation criteria for the CEC 2005 special session on real-parameter optimization," Tech. Rep., Nanyang Technological University, 2005.